



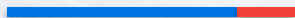
Chaîne de Test Automatisé

De la spécification à

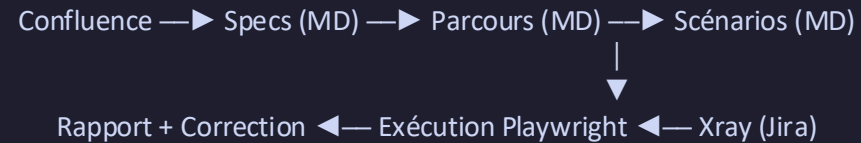
l'exécution

Du besoin métier au rapport de test — une approche outillée de bout en bout

Pilotage qualité assisté par IA



Vue d'ensemble du flux



- **Principe** — chaque étape produit un livrable versionné, traçable, et exploitable par l'étape suivante.

Point clé — la qualité devient objective, mesurable et auditable à chaque maillon de la chaîne

1. Confluence — Source de vérité

- **Exigences métier** — documentées et partagées
- **Référentiel commun** — équipe produit / dev / QA
- **Versionnement natif** — des pages Confluence
- **Extraction automatisée** — via MCP Atlassian (API)

Outil	Usage
Confluence Cloud	Hébergement des spécifications
MCP Atlassian	Extraction automatisée via API

Point clé — Confluence est la source unique de vérité, jamais de copier-coller manuel

2. Spécifications en Markdown

- **Extraction structurée** — depuis Confluence
- **Format portable** — versionnable dans Git
- **Base de dérivation** — pour les parcours de test

```
# Feature: Connexion utilisateur
```

```
## Règles métier
```

- Email obligatoire, format valide
- Mot de passe : 8 caractères min
- Verrouillage après 3 échecs

Point clé — le Markdown garantit la portabilité et la traçabilité via Git (diff, historique, revue)

3. Parcours de test (MD)

- **Décomposition** — des specs en parcours utilisateur
- **Chemins nominaux** — et alternatifs identifiés
- **Couverture fonctionnelle** — tracée et mesurable

Parcours : Connexion réussie

Préconditions

- Compte actif existant

Étapes

1. Accéder à /login
2. Saisir email valide
3. Saisir mot de passe correct
4. Cliquer "Se connecter"

Résultat attendu

- Redirection vers /dashboard

Point clé — un parcours = un scénario métier complet, validable indépendamment

4. Scénarios de test (MD)

- **Formalisation** — en scénarios exécutables
- **Structure tabulaire** — prêt pour injection Xray
- **Données de test** — explicites et reproductibles

Étape	Action	Donnée	Résultat attendu
1	Naviguer vers /login	—	Page login affichée
2	Saisir email	user@test.com	Champ rempli
3	Saisir mot de passe	P@ssw0rd!	Champ masqué rempli
4	Cliquer Connexion	—	Redirect /dashboard

Point clé — le format tabulaire permet l'injection directe dans Xray sans transformation

5. Push dans Xray (Jira)

- **Test Cases** — créés dans Jira/Xray
- **Traçabilité** — rattachement aux exigences
- **Organisation** — en Test Plans et Test Sets
- **Lien bidirectionnel** — Exigence ↔ Test
- **Couverture mesurable** — via dashboards Xray

Outil	Usage
Xray Cloud	Gestion des tests dans Jira
MCP Xray	Création automatisée (API GraphQL)
Script d'injection	MD → Xray via add_manual_test_steps

Point clé — l'injection automatisée garantit la cohérence entre MD source et Xray

6. Scénarios Playwright

- **Automatisation** — des scénarios MD en code
- **Tests E2E** — Chromium, Firefox, WebKit
- **Données paramétrées** — reproductibilité garantie

```
import { test, expect } from '@playwright/test';

test('SC-001 — Connexion nominale', async ({ page }) => {
  await page.goto('/login');
  await page.fill('[data-testid="email"]', 'user@test.com');
  await page.fill('[data-testid="password"]', 'P@ssw0rd!');
  await page.click('[data-testid="submit"]');
  await expect(page).toHaveURL('/dashboard');
});
```

Point clé — le code Playwright est dérivé du scénario MD, assurant la correspondance 1:1

7. Exécution Playwright

- **CI/CD ou local** — lancement automatisé
- **Multi-navigateurs** — multi-devices
- **Traces & vidéos** — pour diagnostic rapide

Outil	Usage
Playwright Test	Runner de tests
GitHub Actions / Azure Pipelines	CI/CD
Playwright Reporter	Rapports HTML/JSON

```
npx playwright test --reporter=html,json
```

Point clé — l'exécution en CI garantit la non-régression à chaque commit

8. Rapport + Correction

- **Analyse** — des résultats (PASS / FAIL)
- **Mise à jour Xray** — statuts synchronisés
- **Bugs Jira** — créés automatiquement si échec
- **Boucle de correction** — re-exécution ciblée

FAIL détecté
↳ Bug créé (Jira)
↳ Fix développée
↳ Re-exécution ciblée
↳ PASS → Clôture bug

Point clé — la boucle de feedback est courte : détection → correction → validation en continu

Cartographie des outils

Étape	Outil principal	Complément
Spécifications	Confluence	MCP Atlassian
Structuration	Markdown + Git	VS Code / OpenCode
Gestion de tests	Xray Cloud	MCP Xray
Automatisation	Playwright	TypeScript
CI/CD	GitHub Actions	Azure Pipelines
Reporting	Playwright HTML	Xray Dashboards
Correction	Jira	MCP Atlassian

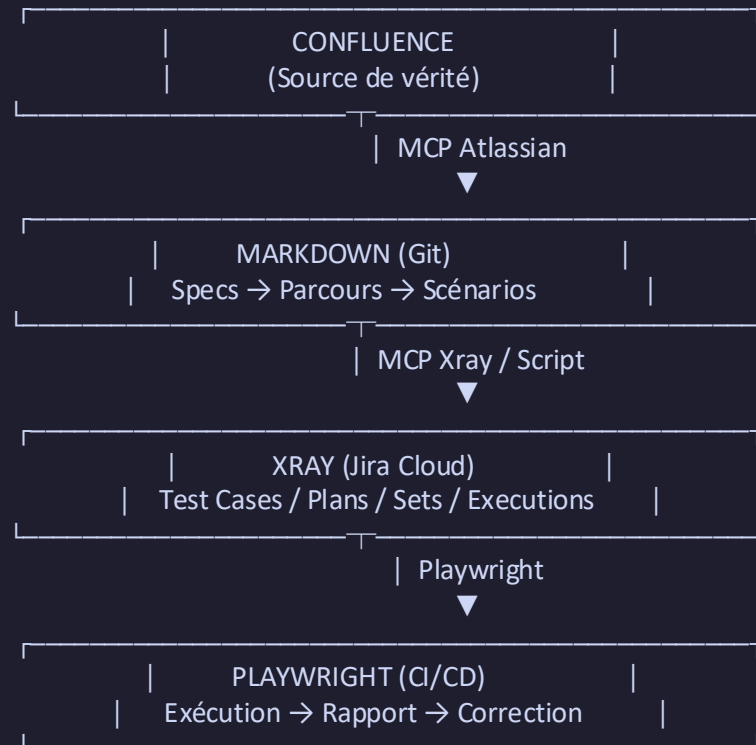
Point clé — chaque outil est spécialisé et interconnecté via MCP et CI/CD

Bénéfices de l'approche

- **Traçabilité complète** — Exigence → Parcours → Scénario → Test Case → Exécution → Rapport
- **Automatisation maximale** — Confluence → MD → Xray → Playwright → Rapport (MCP + CI/CD)
- **Versionnement** — Specs et scénarios dans Git, historique traçable
- **Collaboration** — QA, Dev, PO sur le même référentiel, feedback rapide
- **Qualité auditable** — couverture mesurable, décision de release objective

Point clé — la qualité n'est plus une impression mais une donnée mesurable et traçable

Stack technique résumée





Questions ?

Merci

